Enhancing Trust: The Evolution of Trusted Keys in the Linux Kernel

Parthiban

Linumiz

Linumiz

whoami

- Parthiban, Linumiz
  - Embedded Linux and Zephyr RTOS development, consulting, training
  - Embedded Linux development: BSP, u-boot, Linux Kernel, Yocto Project, Buildroot
  - Zephyr: SoC, Board support, drivers
  - www.linumiz.com
- Living in **Berlin**, Germany

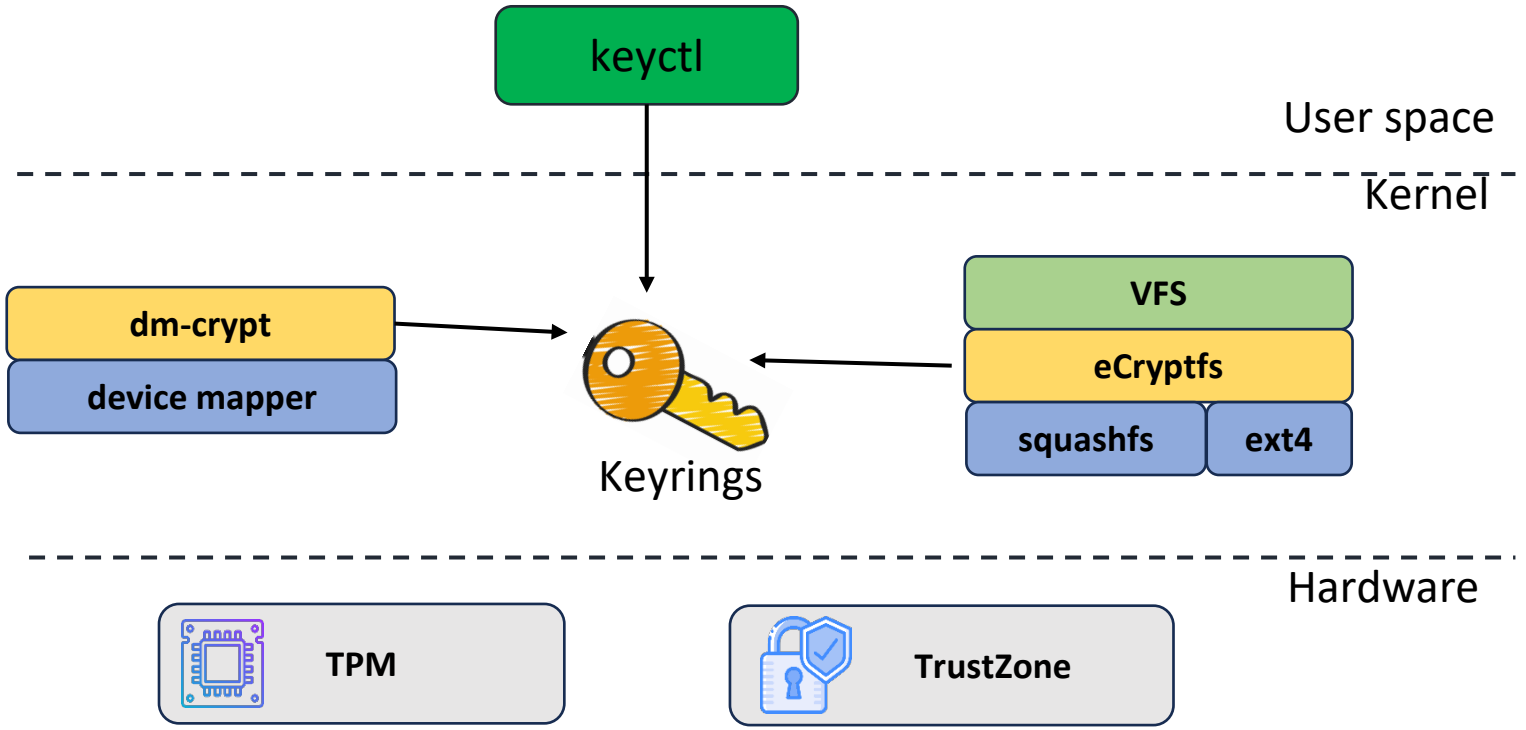
Agenda

- Kernel key retention
- Key types
- Trusted and Encrypted keys
- Trusted keys: Evolution
- Trusted key type: DCP, CAAM, TPM, TEE
- Demo

Kernel key retention

- Manages cryptographic keys in the Linux kernel
- Keys are stored in keyrings for organized access
- Supports various key types: user, trusted, encrypted
- Implements access control based on UID and GID
- Allows setting expiration times for auto key removal
- Provides APIs for adding, revoking, and querying keys

Kernel key retention (Cont.,)



Key

```
struct key {
        key_serial_t            serial;
        struct key_user         *user;
        union {
                time64_t        expiry;
                time64_t        revoked_at;
        };
        kuid_t                  uid;
        kgid_t                  gid;
        key_perm_t              perm;

        union {
                struct {
                        struct key_type *type;
                        char *description;
                        .....
                };
        };

        union {
                union key_payload payload;
                .....
        };
};
```

- **Serial number**: unique integer handle (keyctl show / syscall)
- **Description (name):** Printable string used for search
- **Access rights**: Permissions as like files, user ID & group ID
- **Expiry time**: Future timestamp when set or revoked
- **Payload (data):** Actual key blob / material itself
- **Type**: Key type

Key types

- **User**: Keys store arbitrary data, accessible and modifiable by user space
- **Logon**: Store secrets for kernel use, not readable by user space
- **Big key**: Same as User with 1 MiB payload
- **Keyring**: List of other keys without any payload for itself
  - Link / list of keys or keyring itself
  - analogy to files (keys) and directories (keyrings)
  - Possible operations: Add, Link, Unlink, Clear, Search
- **Encrypted**: AES encrypted, secured by a trusted or user master key
- **Trusted**: Symmetric key with trusted source (usually hardware)

keyctl: Examples

Add

```
# keyctl add user my_user_key "`dd if=/dev/urandom bs=1 count=32`" @u
601819521
# keyctl show
Session Keyring
 388505231 --alswrv      0 65534  keyring: _uid_ses.0
 987750256 --alswrv      0 65534   \_ keyring: _uid.0
 601819521 --alswrv      0     0       \_ user: my_user_key
# keyctl print 601819521
604ac195a3720e9c14c0eb0590099a6cce829e5dae5672a7dbb1c242a5bbc6cc
```

Serial number

Key

Revoke

```
# keyctl revoke 601819521
#
# keyctl clear @u
# keyctl show
Session Keyring
 388505231 --alswrv      0 65534  keyring: _uid_ses.0
 987750256 --alswrv      0 65534   \_ keyring: _uid.0
```

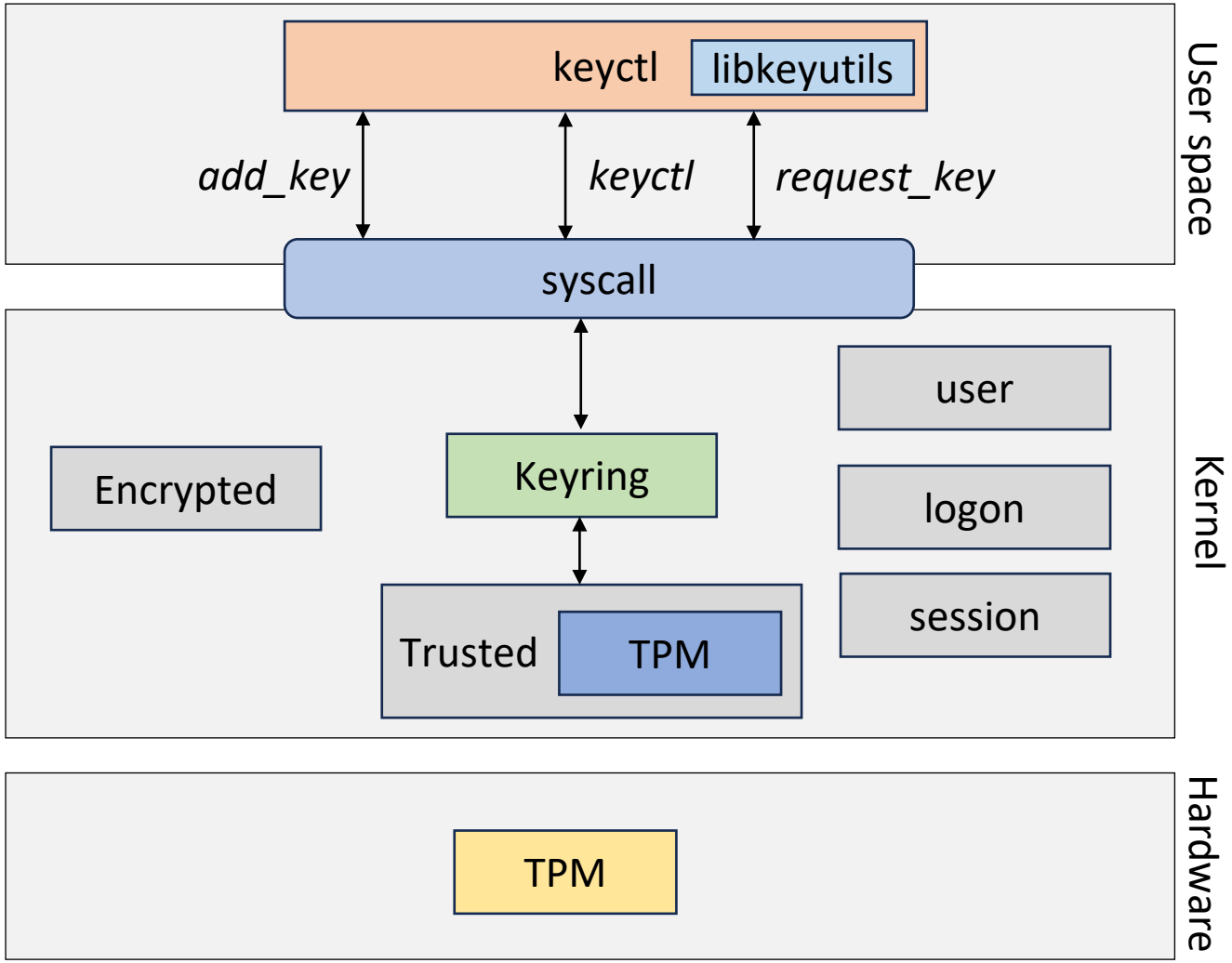
Trusted and Encrypted keys

- Trusted and Encrypted keys are symmetric kernel-created
- User sees only encrypted blobs, stored in hex ASCII
- Trusted Keys need a secure Trust Source for protection
- Encrypted Keys work on any system without special hardware
- Trust Source security depends on implementation and use case
- Integrity of user-level blobs is always verified.

Trust source

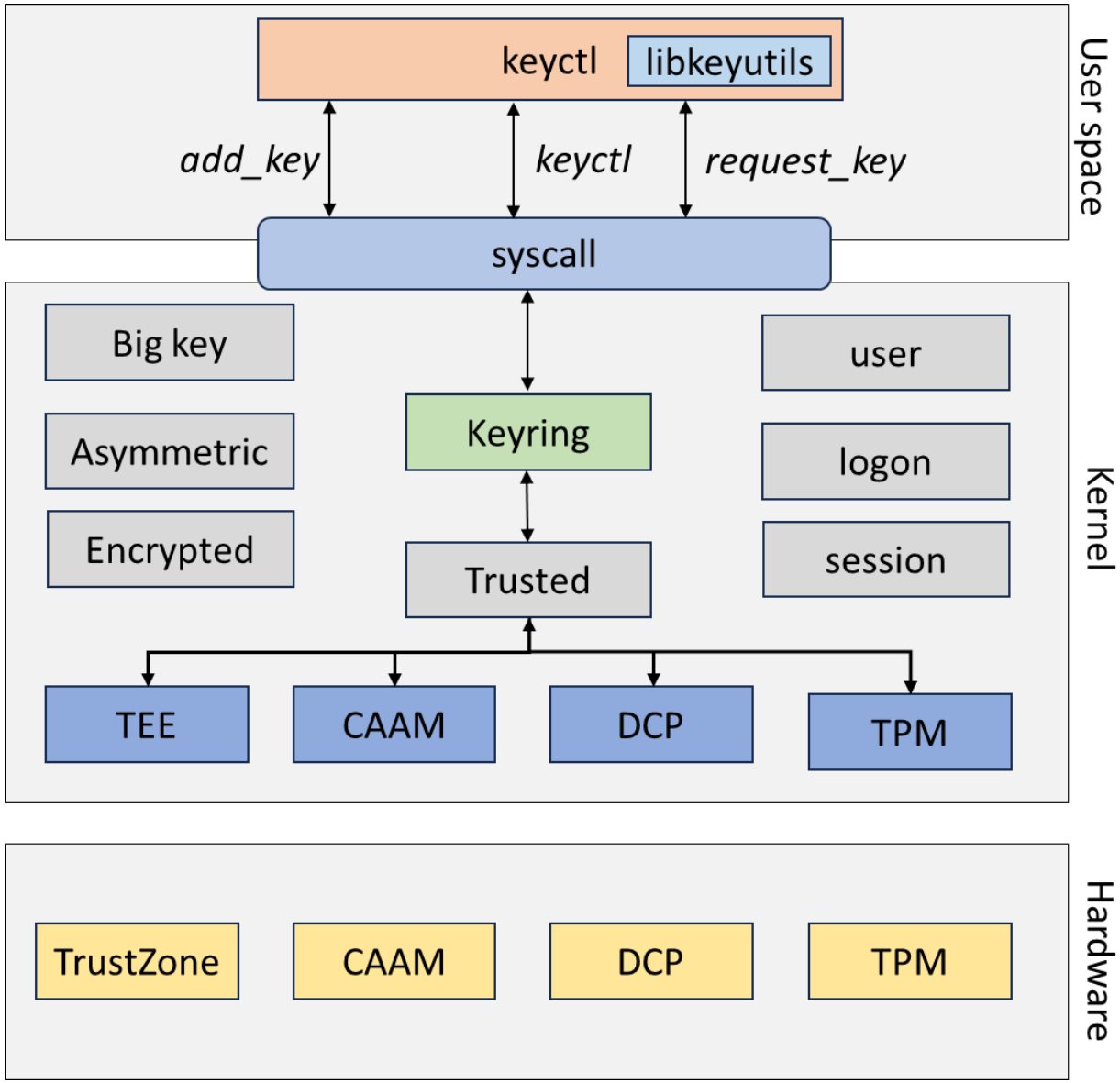
- Root of trust for storage
- Execution isolation
- Platform integrity state

Trusted keys: Evolution (before 5.4)



- Trusted tightly bound to TPM
- Adding additional hardware trust source wasn't possible

Trusted keys: Evolution (after 5.4)



- Trusted key type is abstracted to add additional trust source
- Trusted source implements seal,unseal,get_random
- Keyring or Trusted core doesn't need to know Hardware/SoC specific details

key type

```
struct key_type {
        const char *name;

        int (*preparse)(struct key_preparsed_payload *prep);

        int (*instantiate)(struct key *key, struct key_preparsed_payload *prep);

        int (*update)(struct key *key, struct key_preparsed_payload *prep);

        void (*revoke)(struct key *key);

        long (*read)(const struct key *key, char *buffer, size_t buflen);
        .....
};
```

- instantiate: depends on key type. For trusted it is add/load new key
- read: read encrypted hex bytes
- update: key binding

Trusted source types

- Data Co-Processor (DCP)
- Cryptographic Acceleration and Assurance Module (CAAM)
- Trusted Platform Module (TPM)
- Trusted Execution Environment (TEE) - OP-TEE

trusted_key_ops

```
struct trusted_key_ops {
        unsigned char migratable;

        /* Initialize key interface. */
        int (*init)(void);

        /* Seal a key. */
        int (*seal)(struct trusted_key_payload *p, char *datablob);

        /* Unseal a key. */
        int (*unseal)(struct trusted_key_payload *p, char *datablob);

        /* Optional: Get a randomized key. */
        int (*get_random)(unsigned char *key, size_t key_len);

        /* Exit key interface. */
        void (*exit)(void);
};
```

trusted_key_ops (Cont.,)

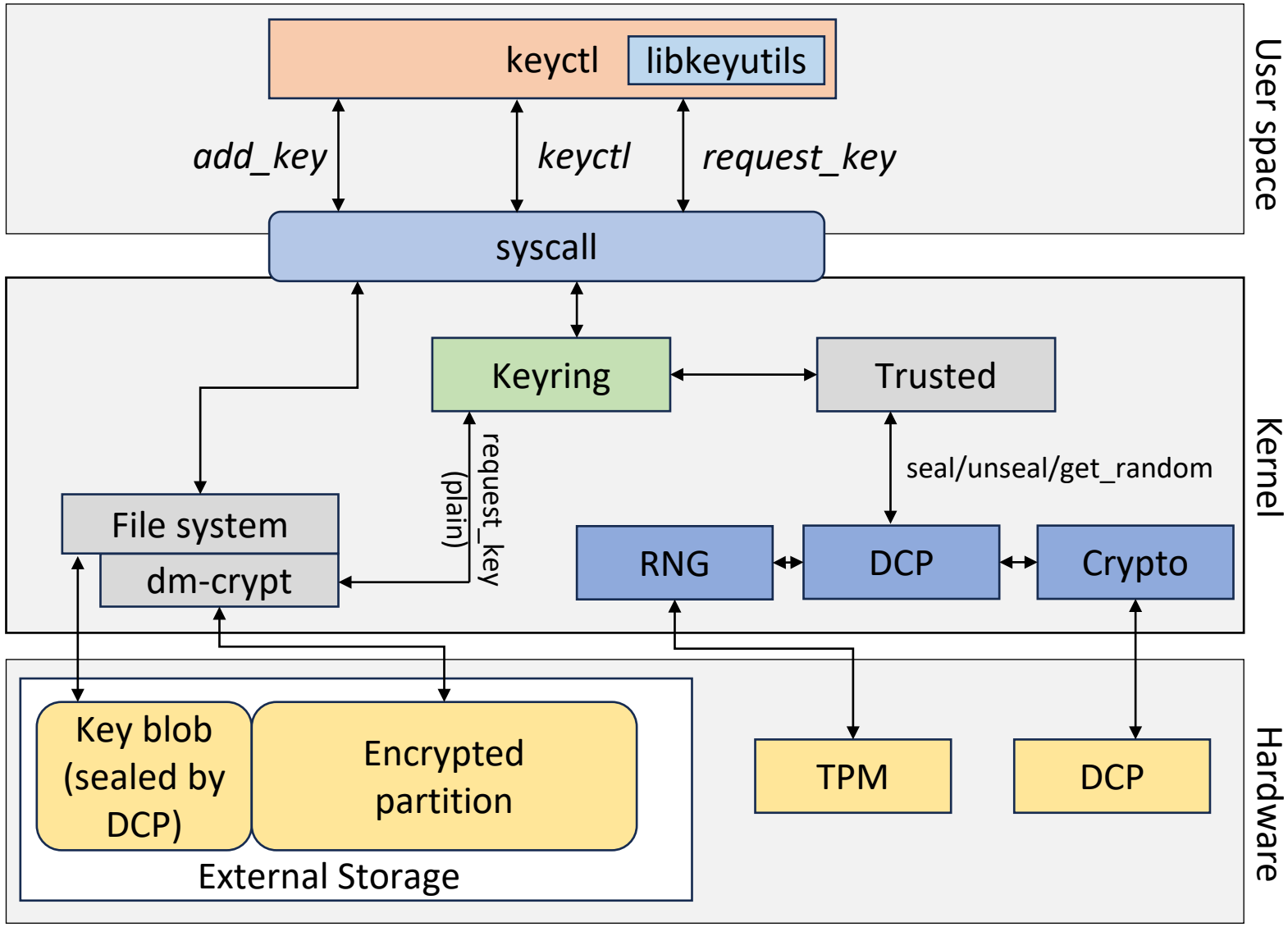
```
static const struct trusted_key_source trusted_key_sources[] = {
#if defined(CONFIG_TRUSTED_KEYS_TPM)
        { "tpm", &trusted_key_tpm_ops },
#endif
#if defined(CONFIG_TRUSTED_KEYS_TEE)
        { "tee", &trusted_key_tee_ops },
#endif
#if defined(CONFIG_TRUSTED_KEYS_CAAM)
        { "caam", &trusted_key_caam_ops },
#endif
#if defined(CONFIG_TRUSTED_KEYS_DCP)
        { "dcp", &dcp_trusted_key_ops },
#endif
};
```

- seal: generates an encrypted key
- unseal: stores the key
- *get_random*: get random key with size. Source of random can be selected using kernel command line *trusted.rng*

i.MX Data Co-Processor (DCP)

- Hardware crypto accelerator
- Supports AES-128 (ECB and CBC modes)
- Hash computation using sha256
- Internal Memory for storing up to four AES-128 keys - accessible only to DCP

Trusted source: DCP



Trusted source: DCP (Cont.,)

- *get_random*
  - default / kernel / rngb
- seal
  - Create 128 bit Blob Encryption Key (BEK)
  - Encrypt trusted key (AES-128-GCM)
  - Encrypt BEK (AES-128-ECB) using hardware OTP
- unseal
  - Decrypt BEK using hardware OTP
  - Decrypt trusted key using BEK (plain)

Trusted source: DCP (Cont.,)

```
struct key {
        key_serial_t            serial;
        struct key_user         *user;
        union {
                time64_t        expiry;
                time64_t        revoked_at;
        };
        kuid_t                  uid;
        kgid_t                  gid;
        key_perm_t              perm;

        union {
                struct {
                        struct key_type *type;
                        char *description;
                        .....
                };
        };

        union {
                union key_payload payload;
                .....
        };
};
```

```
struct dcp_blob_fmt {
        __u8 fmt_version;
        __u8 blob_key[AES_KEYSIZE_128];
        __u8 nonce[AES_KEYSIZE_128];
        __le32 payload_len;
        __u8 payload[];
} __packed;
```

```
struct trusted_key_payload {
        unsigned int key_len;
        unsigned int blob_len;
        unsigned char key[MAX_KEY_SIZE + 1];
        unsigned char blob[MAX_BLOB_SIZE];
};
```

Linumiz

Trusted source: DCP (Cont.,)

Root of trust for storage

- OTP burst to fuses. Accessible only by DCP.
- Two possible options: UNIQUE (default) or OTP key, controlled via module param

  dcp_use_otp_key

Execution isolation

- No hardware blob mechanism
- Seal/Unseal happens in CPU

Platform integrity state

- High Assurance Boot (HAB) must be enabled
- Device must be in closed state (otherwise test key is used)

Demo

Linumiz

Cryptographic Acceleration and Assurance Module (CAAM)

- Cryptographic acceleration and offloading hardware
- Block, stream cipher, hash, PKI algorithms, random number
- Blob mechanism

Trusted source: CAAM

The NXP Cryptographic Acceleration and Assurance Module (CAAM) can be used to protect user-defined data across system reboot:

- When the system is fused and boots into secure state, the master key is a unique never-disclosed device-specific key (OTPMK)
- Random key is encrypted by key derived from master key
- Data is encrypted using the random key
- This random key can now be safely stored in non-volatile memory

On next power-on:

- Random key is loaded into CAAM
- CAAM writes decrypted data either into memory or key register

Trusted source: CAAM

```
struct caam_blob_info {
        void *input;
        size_t input_len;

        void *output;
        size_t output_len;
    ...
};

int caam_process_blob(struct caam_blob_priv *priv,
                      struct caam_blob_info *info, bool encap);
```

Trusted source: CAAM

Root of trust for storage

- Rooted to OTPMK, never disclosed key

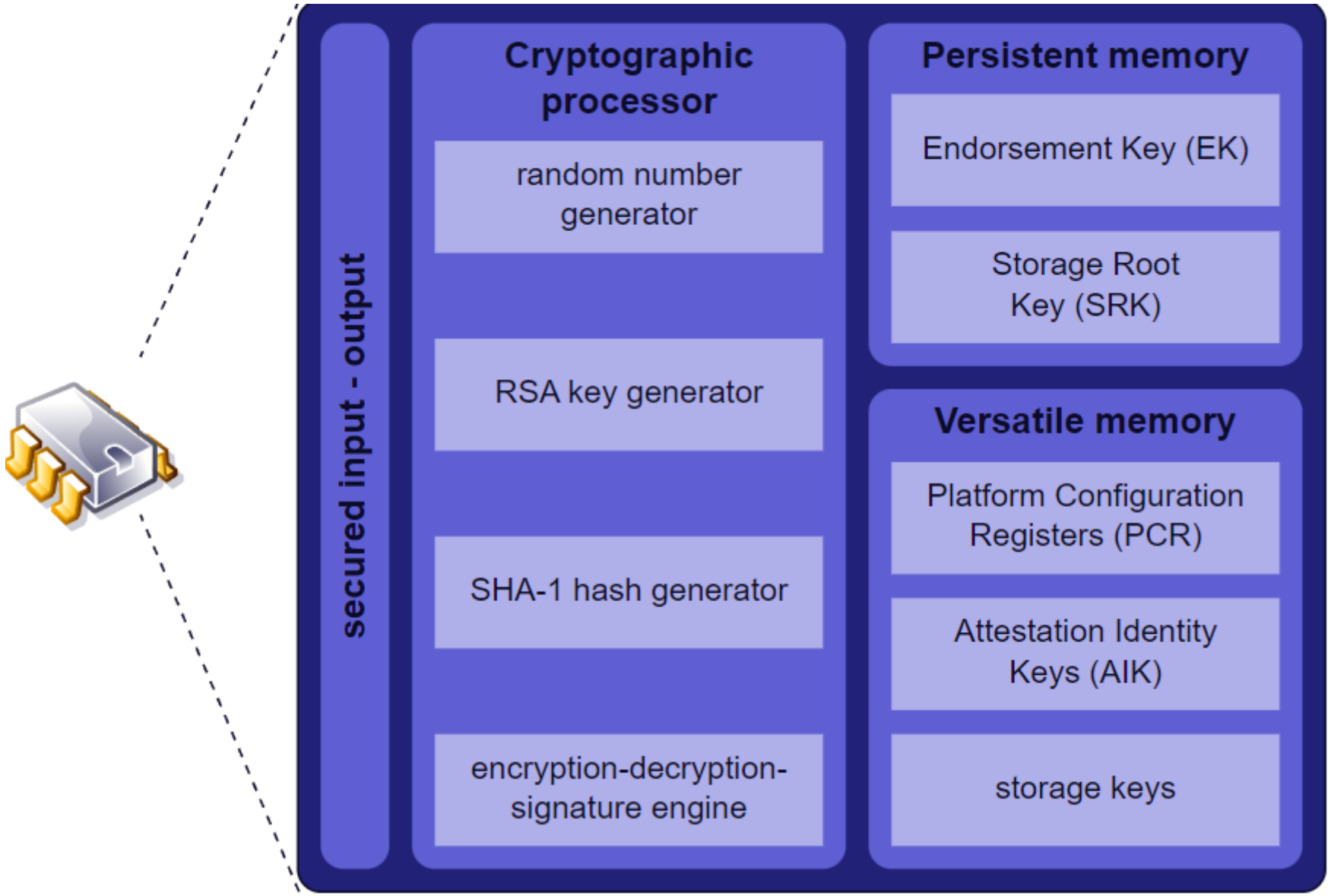
Execution isolation

- Isolated and running in CAAM jobs IP

Platform integrity state

- High Assurance Boot (HAB) must be enabled
- Device must be in closed state (otherwise test key is used)

Trusted source: TPM [1]



Trusted source: TPM

Name	Command code	trusted_key_ops
TPM_CC_GetRandom	0x0000017B	get_random
TPM_CC_Create	0x00000153	seal
TPM_CC_Unseal	0x0000015E	unseal
TPM_CC_Load	0x00000157	

Trusted source: TPM

Root of trust for storage

- Rooted to Storage Root Key (SRK) which never disclosed

Execution isolation

- Isolated and running externally on hardware interfaces (SPI/I2C)

Platform integrity state

- Can be sealed to specific Platform Configuration Register (PCR)
- Updating the key to the corresponding PCR is feasible

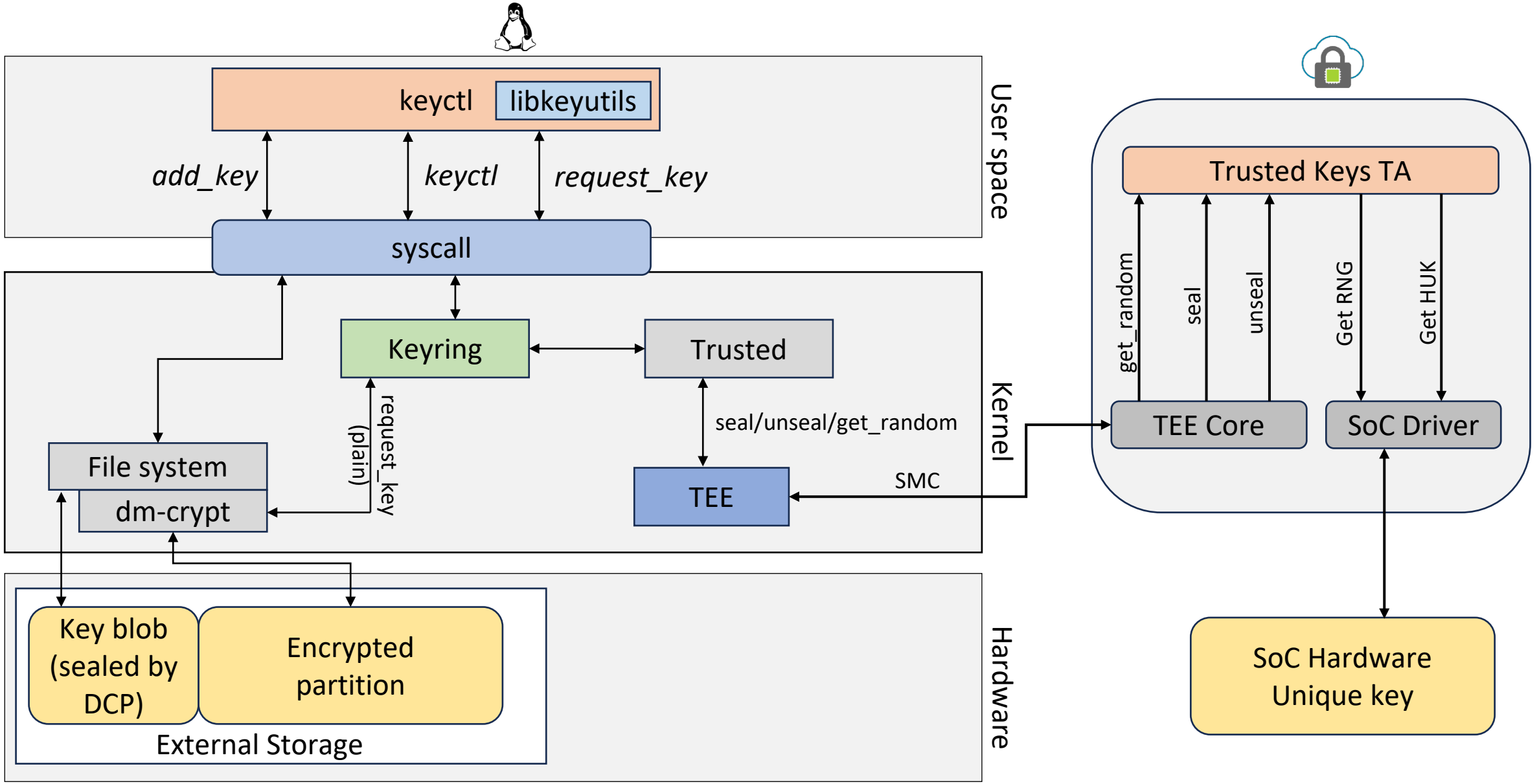
Trusted source: TPM

```
# tpm2_createprimary --hierarchy o -G rsa2048 -c key.ctxt
# tpm2_evictcontrol -c key.ctxt 0x81000001
# modprobe trusted
[162336.444350] Key type trusted registered
# keyctl add trusted tpm_key "new 32 keyhandle=0x81000001" @u
670877384
# keyctl show
Session Keyring
 222622289 --alswrv      0 65534  keyring: _uid_ses.0
 847476474 --alswrv      0 65534   \_ keyring: _uid.0
 670877384 --alswrv      0     0       \_ trusted: tpm_key
# keyctl print 670877384
3081c906066781050a0105a003010101020500810000010430002e0008000b000000400000001
00020925b15b861d03fc57b3d44e6a47778d7b7ce36e3bd34ee8f2f881836577f572b04818000
7e002060ccef611dae55d27e0541007bc9c190a546568fa125b2f47acd9e70d6f9f1a10010506
b038b9560618f23984c4374480c3a2b666f43f049880760030c413c1dc0da71cd83c21850fa05
2afddfc68f65bbab3bcdccca98fdb19ff705ab590e384de9e4b1ef3371425e72fb65dae948a68
d7fef02e603820ccb25aaa6
```

Trusted source: TEE

- Hardware isolation – Trustzone
- Secure operating system - OP-TEE
- SMC by non-secure Rich Execution Environment (REE) - Linux

Trusted source: TEE



Trusted source: TEE

Root of trust for storage

- Rooted to Hardware Unique Key (HUK) is burnt in on-chip fuses, accessible only to TEE

Execution isolation

- Isolated OP-TEE handling it via SMC

Platform integrity state

- Relies on Secure/Trusted boot process for platform integrity

Trusted source: TEE [3]

```
# keyctl add trusted kmk "new 32" @u
[   24.895598] Bad mode in prefetch abort handler detected
[   24.900908] Internal error: Oops - bad mode: 0 [#1] SMP ARM
[   24.906516] Modules linked in: imx_sdma
[   24.910407] CPU: 0 PID: 136 Comm: keyctl Not tainted 6.9.0-g5fd74e5f29e4-dirty #27
[   24.918013] Hardware name: Freescale i.MX6 Ultralite (Device Tree)
[   24.924216] PC is at 0x4000228
[   24.927307] LR is at optee_smccc_smc+0x3c/0x44
[   24.931796] pc : [<04000228>]    lr : [<c073313c>]    psr: 600d01d6
[   24.938087] sp : e0989bd8  ip : e0989c30  fp : c245e980
[   24.943337] r10: 0000020a  r9 : 32000012  r8 : c2367a00
[   24.948585] r7 : 00000000  r6 : 00000000  r5 : 00000000  r4 : 00000000
[   24.955139] r3 : 00000000  r2 : 82d43000  r1 : 00000000  r0 : 32000012
[   24.961695] Flags: nZCv  IRQs off  FIQs off  Mode MON_32  ISA ARM  Segment none
[   24.969034] Control: 10c5387d  Table: 82ba406a  DAC: 00000000
[   24.974803] Register r0 information: non-paged memory
[   24.979892] Register r1 information: NULL pointer
[   24.984626] Register r2 information: non-paged memory
[   24.989706] Register r3 information: NULL pointer
[   24.994439] Register r4 information: NULL pointer
[   24.999171] Register r5 information: NULL pointer
[   25.003903] Register r6 information: NULL pointer
[   25.008635] Register r7 information: NULL pointer
[   25.013368] Register r8 information: slab kmalloc-512 start c2367a00 pointer offset 0 size 512
[   25.022063] Register r9 information: non-paged memory
[   25.027144] Register r10 information: non-paged memory
[   25.032313] Register r11 information: slab kmalloc-64 start c245e980 pointer offset 0 size 64
[   25.040912] Register r12 information: 2-page vmalloc region starting at 0xe0988000 allocated at
kernel_clone+0xa4/0x334
[   25.051763] Process keyctl (pid: 136, stack limit = 0x12664773)
```

On going works

- STM32 hardware unique key (HUK) trusted source
- dm-crypt + dm-init using trust source
  - without initramfs
    - bootloader -> kernel -> initramfs (dm-crypt mount + switch_root) -> RFS

```
dm-crypt,,8,ro, 0 1048576 crypt aes-xts-plain64
3dcc5f324b445ebbad02da2d67325a649e629bdd3dcc5f324b445ebbad02da2d 0 /dev/mmcblk0p2 0 1 allow_discards
```

  - Load in u-boot/bootloader and pass it to kernel driver
  - Often saved to persistent memory, so pass the mtd device or offset

Reference

[1] TPM – Link

[2] Linux Kernel Security: dm-crypt with “trusted keys” using TEE backend –Link

[3] i.MX6ULL: Kernel crash on SMC call - Link

Thank You

Parthiban

parthiban@linumiz.com

Linumiz

Appendix

Linumiz

Trusted keys: DCP Example

```
# zcat /proc/config.gz | grep _KEYS
CONFIG_KEYS=y
# CONFIG_KEYS_REQUEST_CACHE is not set
CONFIG_TRUSTED_KEYS=y
CONFIG_HAVE_TRUSTED_KEYS=y
# CONFIG_TRUSTED_KEYS_TEE is not set
# CONFIG_TRUSTED_KEYS_CAAM is not set
CONFIG_TRUSTED_KEYS_DCP=y
# CONFIG_ENCRYPTED_KEYS is not set
```

```
# cat /proc/cmdline
console=ttymxc0,115200n8 trusted.source=dcp ...
```

Add

```
# keyctl add trusted dcp "new 32" @u
569548642
# keyctl print 569548642
013eea26cc053a86cc806760cce1f05bf705430f39eb84a58bbb215a599fa765f020000000de9
2d4d502472f535da53a002a95bc299e9de63945fec63629df90c6eb6f17a829d5836aa61f8fab
3c67b694eaba3d64
# keyctl pipe 569548642 >dcp.blob
#
# keyctl show
Session Keyring
 998915382 --alswrv      0 65534  keyring: _uid_ses.0
   9348133 --alswrv      0 65534   \_ keyring: _uid.0
 569548642 --alswrv      0     0       \_ trusted: dcp
```

Linumiz

Trusted keys: DCP Example

```
# keyctl add trusted dcp_from_blob "load `cat dcp.blob`" @u
639981142
# keyctl show
Session Keyring
 998915382 --alswrv      0 65534  keyring: _uid_ses.0
   9348133 --alswrv      0 65534   \_ keyring: _uid.0
 639981142 --alswrv      0     0       \_ trusted: dcp_from_blob
# keyctl print 639981142
01afc8945c98249e7fb6818b8eb46759cb05430f39eb84a58bbb215a599fa765f020000000de9
2d4d502472f535da53a002a95bc299e9de63945fec63629df90c6eb6f17a829d5836aa61f8fab
3c67b694eaba3d64
```

Trusted keys: DCP Example

```
# dd if=/dev/zero of=encrypted.img bs=1M count=32 && sync
32+0 records in
32+0 records out
# losetup /dev/loop0 encrypted.img
[  703.445312] loop0: detected capacity change from 0 to 65536
# blockdev --getsz /dev/loop0
65536
# dmsetup -v create encrypted --table "0 $(blockdev --getsz /dev/loop0) crypt
aes-cbc-plain :32:trusted:dcp_from_blob 0 /dev/loop0 0 1 sector_size:512"
Name:              encrypted
State:             ACTIVE
Read Ahead:        256
Tables present:    LIVE
Open count:        0
Event number:      0
Major, minor:      253, 0
Number of targets: 1
# ls -lrt /dev/mapper/
total 0
crw-------    1 root     root       10, 236 Jan  3 19:54 control
brw-------    1 root     root      253,   0 Jan  3 20:07 encrypted
```

Trusted keys: DCP Example

```
# mkfs.ext4 /dev/mapper/encrypted
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 32768 1k blocks and 8192 inodes
Filesystem UUID: 601f5bff-ac99-4fa8-80e0-348a832334af
Superblock backups stored on blocks:
        8193, 24577
Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

# mkdir -p /mnt/encrypted
# mount -t ext4 /dev/mapper/encrypted /mnt/encrypted/
[  785.876507] EXT4-fs (dm-0): mounted filesystem 601f5bff-ac99-4fa8-80e0-
348a832334af r/w with ordered data mode. Quota mode: none.
# ls /mnt/encrypted/
lost+found
# echo "Encrypted using DCP truste source" > /mnt/encrypted/dcp.txt
# cat /mnt/encrypted/dcp.txt
Encrypted using DCP truste source
# sync && umount /mnt/encrypted/ && dmsetup remove encrypted
[  824.507466] EXT4-fs (dm-0): unmounting filesystem 601f5bff-ac99-4fa8-80e0-
348a832334af.
```

Trusted keys: DCP Example

```
# keyctl add trusted dcp_from_blob "load `cat dcp.blob`" @u
658930960
# keyctl show
Session Keyring
 670301575 --alswrv      0 65534  keyring: _uid_ses.0
 550244097 --alswrv      0 65534   \_ keyring: _uid.0
 658930960 --alswrv      0     0       \_ trusted: dcp_from_blob
# keyctl print 658930960
01afc8945c98249e7fb6818b8eb46759cb05430f39eb84a58bbb215a599fa765f020000000de
92d4d502472f535da53a002a95bc299e9de63945fec63629df90c6eb6f17a829d5836aa61f8f
ab3c67b694eaba3d64
# losetup /dev/loop0 ./encrypted.img
[  106.413728] loop0: detected capacity change from 0 to 65536
# dmsetup -v create encrypted --table "0 $(blockdev --getsz /dev/loop0)
crypt aes-cbc-plain :32:trusted:dcp_from_blob 0 /dev/loop0 0 1
sector_size:512"
Name:              encrypted
State:             ACTIVE
Read Ahead:        256
Tables present:    LIVE
Open count:        0
Event number:      0
Major, minor:      253, 0
Number of targets: 1
# mount /dev/mapper/encrypted /mnt/encrypted/
[  119.176999] EXT4-fs (dm-0): mounted filesystem 601f5bff-ac99-4fa8-80e0-
348a832334af r/w with ordered data mode. Quota mode: none.
# cat /mnt/encrypted/dcp.txt
Encrypted using DCP truste source
```

Trusted key type

```
struct key_type key_type_trusted = {
        .name = "trusted",
        .instantiate = trusted_instantiate,
        .update = trusted_update,
        .destroy = trusted_destroy,
        .describe = user_describe,
        .read = trusted_read,
};
EXPORT_SYMBOL_GPL(key_type_trusted);
```